Take-Homes

One take-home for each of the 30 classes. Algorithms & Data Structures, Professor Roger Hunt

Each take-home prepares you for one class. It names the pattern you will meet in that class, explains the idea behind it, assigns a reading, and lists LeetCode practice problems of the same type. The problems you solve in class are different from the practice problems, and you see them only when class begins.

Plan on at least two hours for each take-home. Read the assigned sections first, then attempt the practice problems on LeetCode, and then write your plan on paper. You may use AI tools to learn the pattern, and I expect you to.

Most readings come from the free textbook *Problem Solving with Algorithms and Data Structures using Python*, on Runestone Academy: https://runestone.academy/ns/books/published/pythonds3/index.html. Where the textbook does not cover a pattern, the take-home says so and points you to another source.

Take-Home 1: Big-O, Big-Omega, Big-Theta and Python memory

For Class 1. I post this take-home before the first class, and it is due on paper at the start of Class 1. Plan on at least two hours.

The pattern: complexity analysis

Big-O notation describes how the cost of a solution grows as its input grows. The cost can be time, counted as the number of basic steps the solution takes, or space, counted as the extra memory it needs. To find the Big-O, keep only the term that grows fastest and drop any constant factor: a loop that takes $3n + 5$ steps is $O(n)$. Big-O gives an upper bound, so an $O(n^2)$ solution grows no faster than n^2. Big-Omega, written Ω, gives a lower bound, and Big-Theta, written Θ, gives both at once, so that same loop is $\Theta(n)$. Python adds a second kind of cost, which you will estimate in every class: the actual number of bytes. Every Python integer is an object of about 28 bytes, and a list stores an 8-byte pointer to each of its elements, so a list of n large integers uses about 36n bytes.

How this builds on earlier classes

This is the first class. Every class after it uses these two tools, Big-O and the byte count, to judge a solution.

Reading

- §2.2 What Is Algorithm Analysis?
- §2.3 Big O Notation
- §2.5 Performance of Python Data Structures
- Open Data Structures, §1.3 Mathematical Background, for asymptotic notation
- Python documentation: sys.getsizeof

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
1672	Richest Customer Wealth	Easy
1929	Concatenation of Array	Easy
1295	Find Numbers with Even Number of Digits	Easy

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. For one of the practice problems, count how many times your loop runs as a function of n, and write the Big-O.
8. Predict how many bytes a list of 100,000 integers uses, at 8 bytes for each pointer plus 28 bytes for each integer. Then, outside class, check your prediction in Python. Keep in mind that `sys.getsizeof` of a list counts only the list and its pointers, so you must add the sizes of the integers yourself.

Take-Home 2: Arrays and in-place updates

For Class 2. I hand this out at the end of Class 1, and it is due on paper at the start of Class 2. Plan on at least two hours.

The pattern: a read index and a write index

A Python list is a dynamic array: a block of memory that holds pointers in order and grows when it fills up. Reading or writing any position by its index takes O(1) time, and appending at the end takes O(1) time on average. Inserting or deleting at the front takes O(n) time, because every later element has to shift by one place. Many array problems can be solved in place, inside the input list itself, with two indexes: a read index that scans every element and a write index that marks where the next value you keep should go.

How this builds on earlier classes

Class 1 gave you Big-O and the memory model. In this class you apply them to the most basic structure in Python, the list.

Reading

- §2.6 Lists
- §1.8 Getting Started with Data

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
26	Remove Duplicates from Sorted Array	Easy
27	Remove Element	Easy
66	Plus One	Easy

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. Which list operations in your plan take O(1) time and which take O(n)? Name each one.
8. Could your plan work with O(1) extra space? If so, what would you have to change inside the input list to make it work?

Take-Home 3: Strings and prefix sums

For Class 3. I hand this out at the end of Class 2, and it is due on paper at the start of Class 3. Plan on at least two hours.

The pattern: strings and prefix sums

A Python string cannot be changed after it is created, so every slice and every concatenation builds a new string, and that costs time and memory in proportion to the string's length. A prefix sum is a list of running totals, in which each entry holds the sum of all the values before it. You build it once, in O(n) time, and afterward you can find the sum of any range of values with a single subtraction. Doing work once, up front, so that many later questions become cheap is a trade of memory for time, and you will see that trade throughout the course.

How this builds on earlier classes

Class 2 worked with indexes into lists. A string behaves like a list that you can read but not change, and a prefix-sum list is a list that you build once and then reuse.

Reading

- §2.4 An Anagram Detection Example
- §1.8 Getting Started with Data

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
724	Find Pivot Index	Easy
28	Find the Index of the First Occurrence in a String	Easy
58	Length of Last Word	Easy

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. Which step in your plan copies a string or a list, and what does that copy cost?
8. If your solution had to answer many questions about the same input, what could you compute once, up front?

Take-Home 4: Hash tables

For Class 4. I hand this out at the end of Class 3, and it is due on paper at the start of Class 4. Plan on at least two hours.

The pattern: hashing with sets and dicts

A hash table finds, inserts and deletes a key in O(1) time on average. It does this by turning the key into a position in an array, so that it can go straight to the key instead of searching for it. Python gives you two hash tables. A set remembers which values you have seen. A dict, short for dictionary, remembers something about each value you have seen, such as how many times it appeared or where. Spending O(n) memory on a hash table often turns an $O(n^2)$ search into an O(n) one. Two keys can land on the same position, which is called a collision. A hash table resolves collisions either by chaining, which keeps a short list at each position, or by probing, which tries the next positions until it finds an empty one. Lookups stay O(1) on average only while the table has room, so when it gets too full the table grows and places every key again.

How this builds on earlier classes

Class 3 stored values you had computed in advance in a list, where you find each one by its position. A hash table stores values by key, so you can find a value without knowing where it sits.

Reading

- §5.5 Hashing
- §2.7 Dictionaries
- Open Data Structures, Chapter 5, Hash Tables, for chaining and probing

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
383	Ransom Note	Easy
387	First Unique Character in a String	Easy
219	Contains Duplicate II	Easy

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. What are the keys and the values in your dict, or what does your set hold?
8. What is the Big-O of the brute-force approach, and what does the hash table change?

Take-Home 5: Two pointers and sliding windows

For Class 5. I hand this out at the end of Class 4, and it is due on paper at the start of Class 5. Plan on at least two hours.

The pattern: two pointers, and the window between them

In the two-pointer pattern you keep two indexes and move them according to a rule. In one version the pointers start at the two ends of a list and move toward each other, which takes advantage of a list that is sorted or a string that reads the same in both directions. In the other version both pointers move forward. Each pointer only ever moves in one direction, so together they cross the list at most once, in O(n) time and O(1) extra space.

The stretch between two pointers that move forward is a window. Instead of recomputing everything about the window each time it moves, you update it: you add the element that enters on the right and remove the element that leaves on the left. For a window of fixed size k, this turns an O(nk) scan into an O(n) scan. Other windows change size as they move: the right edge grows the window, and the left edge shrinks it whenever a condition stops holding.

How this builds on earlier classes

Class 2 used a read index and a write index, which are two pointers that both move forward. The prefix sums of Class 3 kept running totals, which is the same idea as the running sum that a window keeps.

Reading

- §3.18 Palindrome Checker
- §2.4 An Anagram Detection Example

Runestone has no section on two pointers or sliding windows. The palindrome checker compares a string's two ends, and the anagram example counts letters, which is what a window keeps track of. Study fixed-size and variable-size windows with your AI tool.

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
977	Squares of a Sorted Array	Easy
167	Two Sum II - Input Array Is Sorted	Medium
3	Longest Substring Without Repeating Characters	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. For two pointers: where does each pointer start, what rule decides which pointer moves, and why does that rule never skip over a correct answer?
8. For a window: what do you keep track of for the window, and what are the two update steps, when an element enters and when one leaves?

Take-Home 6: Stacks

For Class 6. I hand this out at the end of Class 5, and it is due on paper at the start of Class 6. Plan on at least two hours.

The pattern: the stack, and the stack kept in sorted order

A stack works like a pile of plates: you add and remove items only at the top, so the last item in is the first item out. Both operations, called push and pop, take O(1) time. Use a stack when the most recent unfinished item has to be dealt with first, as with matching brackets, undo, backspaces and anything nested inside something else. In Python a list works as a stack: `append` pushes an item, and `pop` removes the item on top.

A monotonic stack keeps its items in sorted order. Before you push a new item, you pop every item that can no longer be the answer to any later question. Each item is pushed once and popped at most once, so a search that looks like $O(n^2)$, such as finding the next larger value for every element of a list, takes only O(n) time.

How this builds on earlier classes

Class 4 used extra memory to remember what you had already seen. A stack also remembers the past, but it keeps it in order, with the newest item always on top.

Reading

- §3.3 Stacks
- §3.5 Implementing a Stack in Python
- §3.6 Simple Balanced Parentheses
- §3.7 Balanced Symbols (A General Case)

Runestone does not cover monotonic stacks. Ask your AI tool to trace one by hand on a small list, and then redo the trace yourself without help.

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
682	Baseball Game	Easy
496	Next Greater Element I	Easy
739	Daily Temperatures	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. What do you push onto the stack, and what event makes you pop?
8. What does the stack hold at the end if the input is valid? For a monotonic stack, do the values increase or decrease from bottom to top, and why?

Take-Home 7: Queues and deques

For Class 7. I hand this out at the end of Class 6, and it is due on paper at the start of Class 7. Plan on at least two hours.

The pattern: the queue, first in and first out

A queue serves items in the order they arrived, like a line at a counter: you add items at the back and remove them from the front. A Python list makes a poor queue, because removing from the front with `pop(0)` takes O(n) time while every other element shifts forward. The `deque` from Python's `collections` module, whose name is short for double-ended queue, adds and removes items at both ends in O(1) time. Queues also model streams of events in which only the recent events matter.

How this builds on earlier classes

The stacks of Class 6 give items back in reverse order, while queues keep them in their original order. Together they are the two basic structures for holding items in a line.

Reading

- §3.10 Queues
- §3.12 Implementing a Queue in Python
- §3.13 Queue Simulation: Hot Potato
- §3.15 Deques

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
225	Implement Stack using Queues	Easy
622	Design Circular Queue	Medium
1700	Number of Students Unable to Eat Lunch	Easy

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. At which end do items enter, and at which end do they leave?
8. How can you build a queue out of stacks, or a stack out of queues, and what does that cost for each operation?

Take-Home 8: Linked lists

For Class 8. I hand this out at the end of Class 7, and it is due on paper at the start of Class 8. Plan on at least two hours.

The pattern: rewiring pointers

A linked list stores each value in its own node, and each node points to the next node. Inserting or deleting at a node you already hold takes O(1) time, but reaching the kth node takes O(k) time, because you must walk there from the head, the first node. Most linked-list problems are solved by walking the list with one or two pointers and carefully changing the `next` pointers. A dummy node placed in front of the head often removes special cases, such as deleting the head itself.

How this builds on earlier classes

The two pointers of Class 5 now walk from node to node instead of from index to index. The queues of Class 7 can also be built out of linked nodes.

Reading

- §3.19 Lists
- §3.21 Implementing an Unordered List: Linked Lists
- §3.23 Implementing an Ordered List

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
21	Merge Two Sorted Lists	Easy
141	Linked List Cycle	Easy
876	Middle of the Linked List	Easy

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. Draw the list as boxes and arrows before and after your change. Which pointers change, and in what order?
8. What happens when the list is empty, when it has one node, and when you have to change the head?

Take-Home 9: Recursion and divide and conquer

For Class 9. I hand this out at the end of Class 8, and it is due on paper at the start of Class 9. Plan on at least two hours.

The pattern: recursion and divide and conquer

A recursive function solves a problem by solving smaller copies of the same problem, and a base case stops the recursion. The running time depends on how many calls the function makes and how much work each call does. The space depends on how deep the calls go, because every call that has not yet returned keeps a frame on the call stack. When the same smaller problem is solved again and again, as in the naive Fibonacci function, recursion takes exponential time unless you store the answers you have already found. Divide and conquer is the form of recursion that splits a problem into parts, solves each part, and combines the results. You describe the cost of a recursive algorithm with a recurrence, an equation that gives the cost of a problem of size n in terms of the cost of smaller problems. Merge sort, which comes next class, satisfies $T(n) = 2T(n/2) + O(n)$, because it solves two halves and then merges them in O(n) time. Drawing the tree of calls and adding up the work level by level solves most recurrences, and the master theorem gives the answer directly for the common form $T(n) = aT(n/b) + f(n)$.

How this builds on earlier classes

Class 8 walked a list one node at a time. Recursion lets a function call itself on the rest of the structure. The call stack that keeps track of those calls is the stack from Class 6.

Reading

- §4.2 What Is Recursion?
- §4.4 The Three Laws of Recursion
- §4.6 Stack Frames: Implementing Recursion
- Jeff Erickson’s notes, “Solving Recurrences”, for recursion trees and the master theorem

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
231	Power of Two	Easy
22	Generate Parentheses	Medium
24	Swap Nodes in Pairs	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. State the base case and the recursive case, in one sentence each.
8. Draw the tree of recursive calls for a small input. How many calls are there, and how deep does the tree go?

Take-Home 10: Sorting I, insertion sort and merge sort

For Class 10. I hand this out at the end of Class 9, and it is due on paper at the start of Class 10. Plan on at least two hours.

The pattern: insertion sort, merge sort, and sorting first

Insertion sort, the simplest good sort for a short or nearly sorted list, takes $O(n^2)$ time in the worst case but only O(n) on a list that is already sorted. No sorting method that works by comparing pairs of elements can do better than O(n log n) time in the worst case. Merge sort reaches that bound: it splits the list in half, sorts each half, and merges the two sorted halves in O(n) time. Merge sort is stable, which means that equal elements keep their original order, and it needs O(n) extra space. Sorting first often makes the rest of a problem easy, because duplicates end up next to each other, pairs can be found with two pointers, and binary search becomes possible.

How this builds on earlier classes

Merge sort has the shape of the recursion from Class 9, and its merge step is the two-pointer scan from Class 5, the same scan that merged two sorted lists in Class 2.

Reading

- §5.6 Sorting
- §5.9 The Insertion Sort
- §5.11 The Merge Sort

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
506	Relative Ranks	Easy
147	Insertion Sort List	Medium
148	Sort List	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. Which sort will you use, and what are its running time and its extra space?
8. Does the order of equal elements matter in your problem? In other words, do you need a stable sort?

Take-Home 11: Sorting II, quicksort and counting sort

For Class 11. I hand this out at the end of Class 10, and it is due on paper at the start of Class 11. Plan on at least two hours.

The pattern: partitioning, and counting instead of comparing

Quicksort picks one element, the pivot, and partitions the list around it, with the smaller elements to its left and the larger ones to its right. It then sorts the two sides by the same method. Partitioning takes O(n) time, so quicksort takes O(n log n) time on average, but a run of bad pivots, such as always picking the smallest element of a list that is already sorted, makes it $O(n^2)$. A random pivot makes that worst case very unlikely, and quicksort sorts in place, with only O(log n) extra space on average for its recursion.

Counting sort does not compare elements at all. When the values are small integers, it counts how many times each value appears and writes the values out in order, in O(n + k) time for k possible values. Radix sort extends the idea to larger numbers by sorting on one digit at a time. Neither is bound by the O(n log n) limit, because that limit applies only to sorting by comparisons.

How this builds on earlier classes

Class 10 sorted by splitting the list in half and merging the halves, with O(n) extra space. Quicksort does its main work, the partition, before it recurses instead of after, and it needs no merge. The partition step moves two pointers, as in Class 5.

Reading

- §5.12 The Quicksort
- Open Data Structures, §11.2 Counting Sort and Radix Sort

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
274	H-Index	Medium
215	Kth Largest Element in an Array	Medium
164	Maximum Gap	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. Which element will your partition use as the pivot, and what happens to the running time if the pivot is always the smallest element?
8. If you count instead of comparing, how many counters do you need, and what does the range of the values cost in time and in memory?

Take-Home 12: Binary search

For Class 12. I hand this out at the end of Class 11, and it is due on paper at the start of Class 12. Plan on at least two hours.

The pattern: halving a sorted range

On a sorted list, binary search compares the target with the middle element and throws away the half that cannot contain it. That takes O(log n) time and O(1) space. The hard part is the details: which half to keep, whether the middle element stays in the range, and when to stop. Practice writing the loop around an invariant, a statement that is true at the top of every pass through the loop, such as “if the answer exists, it lies between lo and hi.”

How this builds on earlier classes

Class 10 produced sorted data, and sorted data is what binary search needs. The halving you saw in the recursion of Class 9 becomes a loop here.

Reading

- §5.4 The Binary Search
- Python documentation: bisect

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
35	Search Insert Position	Easy
69	Sqrt(x)	Easy
33	Search in Rotated Sorted Array	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. Write your invariant in words: what is true about lo and hi at the top of every pass through your loop?
8. Trace a list with one element and a list with two elements through your loop. This checks for off-by-one errors, mistakes in which an index ends up one too high or one too low.

Take-Home 13: Trees and traversal

For Class 13. I hand this out at the end of Class 12, and it is due on paper at the start of Class 13. Plan on at least two hours.

The pattern: depth-first and breadth-first search on a binary tree

A node in a binary tree holds a value and points to at most two children, a left child and a right child. Depth-first search, or DFS, goes as deep as it can before backing up, and you write it as a recursion on the left and right subtrees. It comes in three orders, called preorder, inorder and postorder, which differ in when you visit the node itself. Breadth-first search, or BFS, visits the tree one level at a time and uses a queue. Both searches visit every node once, so both take O(n) time. DFS needs extra space in proportion to the tree's height, and BFS needs extra space in proportion to the tree's widest level.

How this builds on earlier classes

The recursion of Class 9 matches the shape of a tree, in which every subtree is itself a smaller tree. The queues of Class 7 drive the level-by-level traversal.

Reading

- §6.2 Examples of Trees
- §6.3 Vocabulary and Definitions
- §6.6 Nodes and References
- §6.8 Tree Traversals

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
226	Invert Binary Tree	Easy
111	Minimum Depth of Binary Tree	Easy
199	Binary Tree Right Side View	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. What does your function return for an empty tree, and what does it return for a leaf, a node with no children?
8. Is DFS or BFS the better fit for this problem? How much extra space does each one need on a tree that is a single long chain?

Take-Home 14: Binary search trees

For Class 14. I hand this out at the end of Class 13, and it is due on paper at the start of Class 14. Plan on at least two hours.

The pattern: the binary-search-tree property

In a binary search tree, every value in a node's left subtree is smaller than the node's value, and every value in its right subtree is larger. Search, insert and delete each follow a single path from the root down toward a leaf, so they take O(h) time, where h is the height of the tree. That is O(log n) when the tree is balanced and O(n) when the tree has become a single chain. An inorder traversal of a binary search tree visits the values in sorted order.

How this builds on earlier classes

The traversals of Class 13 visit every node. The binary search of Class 12 halves a sorted list, and a binary search tree stores the same halving in its pointers.

Reading

- §6.12 Binary Search Trees
- §6.13 Search Tree Operations
- §6.14 Search Tree Implementation
- §6.15 Search Tree Analysis

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
230	Kth Smallest Element in a BST	Medium
235	Lowest Common Ancestor of a Binary Search Tree	Medium
450	Delete Node in a BST	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. Use the binary-search-tree property to decide which subtree to visit and which to skip.
8. What does an inorder traversal of a valid binary search tree produce, and how can you use that in your solution?

Take-Home 15: Balanced search trees

For Class 15. I hand this out at the end of Class 14, and it is due on paper at the start of Class 15. Plan on at least two hours.

The pattern: keeping a tree's height at O(log n)

A binary search tree is fast only while it is short. Search, insert and delete take O(h) time, and a tree built from values that arrive in sorted order becomes a chain, with a height h of n. A balanced search tree keeps its height at O(log n), whatever order the values arrive in. An AVL tree, named after its inventors, Georgy Adelson-Velsky and Evgenii Landis, stores at each node a balance factor: the height of the left subtree minus the height of the right subtree. It keeps every balance factor at −1, 0 or 1. When an insertion pushes a balance factor to 2 or −2, the tree repairs itself with one or two rotations, local moves that change a few pointers, restore the balance, and keep the values in order. Red-black trees reach the same goal with a looser rule; Java's `TreeMap`, for example, is a red-black tree.

How this builds on earlier classes

Class 14 showed that every operation on a binary search tree follows one path, so its cost is the height of the tree. This class keeps that height small. The recursion on subtrees from Class 13 computes the heights.

Reading

- §6.16 Balanced Binary Search Trees
- §6.17 AVL Tree Performance
- §6.18 AVL Tree Implementation

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
108	Convert Sorted Array to Binary Search Tree	Easy
109	Convert Sorted List to Binary Search Tree	Medium
669	Trim a Binary Search Tree	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. Insert the values 1 through 7, in that order, into an AVL tree by hand, and draw the tree after each rotation.
8. Why does keeping every balance factor between −1 and 1 keep the height at O(log n)? Answer with a small example.

Take-Home 16: Heaps, priority queues and heap sort

For Class 16. I hand this out at the end of Class 15, and it is due on paper at the start of Class 16. Plan on at least two hours.

The pattern: the heap

A binary heap keeps the smallest element at the root of a tree that is filled level by level and stored in a list; a max-heap keeps the largest element there instead. Adding an element and removing the smallest each take O(log n) time, and looking at the smallest takes O(1) time. Use a heap when you repeatedly need the current best item: the top k items, the kth largest item, or the next item from several sorted streams. Python's `heapq` module gives you a min-heap, which keeps the smallest element at the root. Heap sort uses a heap to sort: it builds a heap from the list in O(n) time and then removes the largest element n times, for O(n log n) time in all, in place.

How this builds on earlier classes

The binary search trees of Class 14 keep every value in order through pointers. A heap keeps only a partial order, and it keeps it in a plain list. Sorting, from Class 10, costs O(n log n) time, while a heap that never holds more than k items lets you find the top k in O(n log k) time.

Reading

- §6.9 Priority Queues with Binary Heaps
- §6.10 Binary Heap Operations
- §6.11 Binary Heap Implementation
- Python documentation: heapq
- Open Data Structures, §11.1 Comparison-Based Sorting, for heap sort

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
1046	Last Stone Weight	Easy
1337	The K Weakest Rows in a Matrix	Easy
973	K Closest Points to Origin	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. Do you need a min-heap or a max-heap? Python has only a min-heap, so how will you get the behavior of a max-heap? One way is to store each value with its sign reversed.
8. How many items does your heap hold at most, and why does that size give the running time you claim?

Take-Home 17: Graphs and depth-first search

For Class 17. I hand this out at the end of Class 16, and it is due on paper at the start of Class 17. Plan on at least two hours.

The pattern: depth-first search on graphs and grids

A graph is a set of vertices, also called nodes, joined by edges. You can store a graph as an adjacency list, which is a dict that maps each vertex to the list of its neighbors. A graph can also be stored as an adjacency matrix, an n × n grid in which entry [i][j] records the edge from i to j. A matrix answers "is there an edge from i to j?" in O(1) time but takes $O(V^2)$ space, so it suits graphs in which most pairs are joined, while an adjacency list takes O(V + E) space and suits the rest. You can also treat a grid as a graph in which each cell is a vertex with up to four neighbors. Depth-first search follows one path as far as it goes before it backs up and tries another, and a visited set keeps it from going around a cycle forever. Depth-first search takes O(V + E) time, where V is the number of vertices and E the number of edges, and it needs O(V) extra space.

How this builds on earlier classes

Depth-first search on a tree, from Class 13, is the same search on a graph that has no cycles. The visited set uses the hashing of Class 4 to break cycles.

Reading

- §7.2 Vocabulary and Definitions
- §7.4 An Adjacency Matrix
- §7.5 An Adjacency List
- §7.15 General Depth-First Search
- §7.16 Depth-First Search Analysis

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
463	Island Perimeter	Easy
695	Max Area of Island	Medium
133	Clone Graph	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. What are the vertices, what are the edges, and how do you list the neighbors of one vertex?
8. At what point do you mark a cell as visited, and what goes wrong if you mark it too late?

Take-Home 18: Breadth-first search

For Class 18. I hand this out at the end of Class 17, and it is due on paper at the start of Class 18. Plan on at least two hours.

The pattern: breadth-first search and the shortest path in edges

Breadth-first search explores every vertex one edge away from the start, then every vertex two edges away, and so on, using a queue. The first time it reaches a vertex, it has found the shortest path to that vertex, measured in number of edges. A multi-source breadth-first search starts with several vertices in the queue at once and spreads outward from all of them together. Breadth-first search takes O(V + E) time and O(V) space.

How this builds on earlier classes

The depth-first search of Class 17 used a stack of calls. Breadth-first search replaces that stack with the queue from Class 7, and in exchange it finds shortest paths.

Reading

- §7.7 The Word Ladder Problem
- §7.9 Implementing Breadth-First Search
- §7.10 Breadth-First Search Analysis

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
542	01 Matrix	Medium
1091	Shortest Path in Binary Matrix	Medium
752	Open the Lock	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. What counts as one step, and what is the starting state, or what are the starting states?
8. Do you mark a state as visited when you add it to the queue or when you take it out? Why does it matter?

Take-Home 19: Topological sort

For Class 19. I hand this out at the end of Class 18, and it is due on paper at the start of Class 19. Plan on at least two hours.

The pattern: ordering tasks that depend on each other

A directed graph is a graph whose edges point one way. A topological order lists the vertices of a directed graph so that every edge goes from an earlier vertex to a later one. Getting dressed is an example: socks come before shoes, so any workable order of dressing is a topological order. Such an order exists only when the graph has no cycle. The number of edges that point into a vertex is called its in-degree. Kahn's algorithm builds the order by repeatedly removing a vertex whose in-degree is 0, and it keeps those vertices in a queue. A second method, based on depth-first search, orders the vertices by the time each one finishes. Both methods take O(V + E) time, and both detect a cycle when one exists.

How this builds on earlier classes

The breadth-first search of Class 18, with its queue, becomes Kahn's algorithm. The graphs of Class 17 now have edges with a direction.

Reading

- §7.17 Topological Sorting

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
1557	Minimum Number of Vertices to Reach All Nodes	Medium
802	Find Eventual Safe States	Medium
2115	Find All Possible Recipes from Given Supplies	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. Which way does each edge point: from the thing that must come first to the thing that depends on it, or the other way? Write down the rule for computing each vertex’s in-degree.
8. How will your algorithm notice that the graph has a cycle?

Take-Home 20: Shortest paths with Dijkstra's algorithm

For Class 20. I hand this out at the end of Class 19, and it is due on paper at the start of Class 20. Plan on at least two hours.

The pattern: Dijkstra's algorithm with a heap

Dijkstra's algorithm finds the cheapest path from a starting vertex to every other vertex, in a graph whose edges have costs, called weights, that are never negative. It keeps a min-heap of pairs (distance, vertex). At each step it takes out the closest vertex it has not yet finished, and it tries to improve the distance to each of that vertex's neighbors by going through it. With a binary heap this takes O((V + E) log V) time and O(V + E) space. Many problems hide Dijkstra's algorithm by defining the distance of a path as something other than a sum of edge costs, such as the largest single step along the path.

How this builds on earlier classes

The breadth-first search of Class 18 is Dijkstra's algorithm on a graph in which every edge costs 1. The heap from Class 16 replaces the plain queue of breadth-first search.

Reading

- §7.19 Shortest Path Problems
- §7.20 Dijkstra's Algorithm
- §7.21 Analysis of Dijkstra's Algorithm

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
1514	Path with Maximum Probability	Medium
1976	Number of Ways to Arrive at Destination	Medium
778	Swim in Rising Water	Hard

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. What is the distance you are minimizing, and how do you combine the distance so far with the cost of the next edge?
8. Why can the same vertex come out of the heap more than once, and how do you skip the outdated entries?

Take-Home 21: Shortest paths II, Bellman-Ford and Floyd-Warshall

For Class 21. I hand this out at the end of Class 20, and it is due on paper at the start of Class 21. Plan on at least two hours.

The pattern: relaxing every edge, and trying every stopover

Dijkstra's algorithm fails when an edge can have a negative weight, and it cannot respect a limit on the number of edges in a path. The Bellman-Ford algorithm handles both. It relaxes every edge, which means that it tries to improve the distance to each edge's far end by going through its near end, and it repeats that V − 1 times. After round r it knows the cheapest path of at most r edges, so V − 1 rounds find every shortest path, in O(V × E) time, and a further round that still improves a distance reveals a negative cycle.

When you need the distance between every pair of vertices, the Floyd-Warshall algorithm fills a V × V table of distances and lets each vertex in turn serve as a stopover: if going from i to j through k is shorter than the best route known so far, it records the shorter distance. It takes $O(V^3)$ time and $O(V^2)$ space. Floyd-Warshall is dynamic programming, with the stopover k as the stage.

How this builds on earlier classes

The Dijkstra's algorithm of Class 20 found the shortest paths from one starting vertex when no weight is negative. Bellman-Ford gives up some speed to handle negative weights and limits on the number of edges. Floyd-Warshall stores its answers in a table, as the dynamic programming of Classes 27 to 29 will.

Reading

- Chapter 8, "Shortest Paths," of Jeff Erickson's *Algorithms*, for Bellman-Ford
- Chapter 9, "All-Pairs Shortest Paths," of the same book, for Floyd-Warshall

Runestone covers Dijkstra's algorithm but neither of these, so both readings come from Erickson's free textbook.

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
1462	Course Schedule IV	Medium
399	Evaluate Division	Medium
2976	Minimum Cost to Convert String I	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. How many rounds does your Bellman-Ford need, and why does each round read the distances from the round before rather than the ones it is writing?
8. In Floyd-Warshall, what does the table hold once vertex k has served as a stopover? State it in one sentence.

Take-Home 22: Union-find

For Class 22. I hand this out at the end of Class 21, and it is due on paper at the start of Class 22. Plan on at least two hours.

The pattern: union-find, also called disjoint sets

Union-find keeps track of items that are divided into groups. It supports two operations: find, which tells you which group an item belongs to, and union, which merges two groups into one. Each group is stored as a tree, and the root of the tree serves as the group's label. Two improvements make both operations fast. Path compression points every item on the path of a find straight at the root, and union by size hangs the smaller tree under the larger one. With both improvements, each operation takes very nearly O(1) time on average over a long run of operations; the exact bound uses the inverse Ackermann function, which stays below 5 for any input you could store. Use union-find to count connected groups and to find the edge that closes a cycle. Class 23 uses it to build minimum spanning trees.

How this builds on earlier classes

Class 17 found connected groups with depth-first search. Union-find keeps track of the groups as edges arrive one at a time, and it stores every group's tree in one plain list, in which each item's entry names its parent.

Reading

- VisuAlgo: Union-Find Disjoint Sets
- Jeff Erickson's notes, "Disjoint Sets"

Runestone has no section on union-find. Watch the VisuAlgo animation, and then read Erickson's notes on path compression and union by size.

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
990	Satisfiability of Equality Equations	Medium
1319	Number of Operations to Make Network Connected	Medium
721	Accounts Merge	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. What are the items, and what does it mean in this problem for two items to be in the same group?
8. Write find with path compression in four lines, and say what changes without path compression.

Take-Home 23: Minimum spanning trees

For Class 23. I hand this out at the end of Class 22, and it is due on paper at the start of Class 23. Plan on at least two hours.

The pattern: connecting everything at the lowest cost

A spanning tree of a connected graph is a set of V − 1 edges that connects every vertex without forming a cycle. A minimum spanning tree is a spanning tree whose total weight is as small as possible: the cheapest way to wire every building or to connect every city. Two greedy algorithms find it. Kruskal's algorithm sorts the edges by weight and keeps each edge that joins two separate groups, using union-find to tell whether the two ends are already connected; it takes O(E log E) time. Prim's algorithm grows one tree from a starting vertex and always adds the cheapest edge leaving the tree. With a heap it takes O(E log V) time, and with a simple scan for the cheapest edge it takes $O(V^2)$, which is better when nearly every pair of vertices is joined. Both algorithms are correct for the same reason: if you divide the vertices into two groups in any way, the cheapest edge between the groups belongs to some minimum spanning tree.

How this builds on earlier classes

Kruskal's algorithm needs the union-find of Class 22 and the sorting of Class 10. Prim's algorithm looks like the Dijkstra's algorithm of Class 20, with the weight of one edge in place of the length of a whole path, and it uses the heap of Class 16.

Reading

- §7.22 Prim's Spanning Tree Algorithm
- Chapter 7, "Minimum Spanning Trees," of Jeff Erickson's *Algorithms*, for Kruskal's algorithm and the reason both algorithms work

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
2492	Minimum Score of a Path Between Two Cities	Medium
1579	Remove Max Number of Edges to Keep Graph Fully Traversable	Hard
1697	Checking Existence of Edge Length Limited Paths	Hard

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. Run Kruskal’s algorithm by hand on a graph of five vertices. In what order do you consider the edges, and which do you skip?
8. Why is the cheapest edge leaving the tree always safe for Prim’s algorithm to add? Answer in one or two sentences.

Take-Home 24: Backtracking, P and NP

For Class 24. I hand this out at the end of Class 23, and it is due on paper at the start of Class 24. Plan on at least two hours.

The pattern: choose, explore, undo

Backtracking builds an answer one choice at a time. At each step you make a choice, explore every way to finish the answer from there, and then undo the choice and try the next one. The undoing is the backtrack. You prune a branch, which means you stop exploring it, as soon as you can tell that it cannot lead to a valid answer. The running time is the number of nodes in the tree of choices, which is usually exponential. The space is the depth of the recursion plus the partial answer you are building.

Many problems that call for backtracking, such as finding a route that visits every city exactly once, have no known algorithm that runs in polynomial time. The class P holds the problems that can be solved in polynomial time, such as $O(n^2)$ or $O(n^3)$. The class NP holds the problems whose answers can be checked in polynomial time: given a proposed route, you can quickly confirm that it visits every city once. Every problem in P is also in NP, and whether P equals NP is the most famous open question in computer science. The hardest problems in NP are called NP-complete, because a fast algorithm for any one of them would give a fast algorithm for all of them. When a problem is NP-complete, a careful search with good pruning, or an approximation, is usually the best you can do.

How this builds on earlier classes

The recursion of Class 9 gives backtracking its shape. The depth-first search of Class 17 marked each cell as visited and left it marked; backtracking adds one step, unmarking a cell when you back out of it, so that a later path can use it again.

Reading

- §4.9 Complex Recursive Problems
- §4.11 Exploring a Maze
- §7.11 The Knight's Tour Problem
- Chapter 12, "NP-Hardness," of Jeff Erickson's *Algorithms*, for P, NP and NP-completeness

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
46	Permutations	Medium
39	Combination Sum	Medium
79	Word Search	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. What is one choice, what is the partial answer, and when is the answer complete?
8. Which branches can you cut early? State your pruning rule.
9. Name one problem from this course that is in P and one well-known problem that is NP-complete. What makes the second one hard?

Take-Home 25: Tries

For Class 25. I hand this out at the end of Class 24, and it is due on paper at the start of Class 25. Plan on at least two hours.

The pattern: the trie, or prefix tree

A trie stores strings one character at a time along the edges of a tree, so all the strings that share a prefix share a path from the root. Inserting a word, searching for a word, and checking whether any stored word starts with a given prefix each take O(L) time for a word of length L, no matter how many words the trie holds. Combined with depth-first search, a trie lets you explore many words at once.

How this builds on earlier classes

A trie combines three earlier ideas: the trees of Class 13, the dicts of Class 4, which hold each node's children, and the depth-first search of Class 17. Searches that combine a trie with the backtracking of Class 24 can stop a branch the moment its letters stop matching any prefix in the trie.

Reading

- §6.3 Vocabulary and Definitions
- §6.6 Nodes and References
- §2.7 Dictionaries

Runestone has no section on tries. Read the sections above for the vocabulary of trees, then ask your AI tool to explain tries, and trace an insert by hand yourself.

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
720	Longest Word in Dictionary	Medium
648	Replace Words	Medium
1268	Search Suggestions System	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. What does each node store, such as its children and a flag that marks the end of a word, and how do you represent the children?
8. In the worst case, how many nodes can a trie hold if it stores m words of at most L letters each?

Take-Home 26: Greedy algorithms and intervals

For Class 26. I hand this out at the end of Class 25, and it is due on paper at the start of Class 26. Plan on at least two hours.

The pattern: the greedy choice, and intervals

A greedy algorithm makes the choice that looks best at each step and never goes back on it. A greedy algorithm is correct only when you can argue that the local choice cannot make the final answer worse. The usual argument is an exchange argument: you show that any best answer can be changed, one step at a time, into the greedy answer without getting worse. Greedy algorithms are usually fast, because they need only a sort or a single scan, which means O(n log n) or O(n) time. Always test your rule on a small input that might break it before you trust it.

Intervals are a common setting for greedy choices. Most interval problems start by sorting the intervals by their start or by their end. A single scan from left to right, called a sweep, then merges intervals that overlap, or picks as many intervals as possible that do not overlap. For that second task the classic greedy rule is to choose, again and again, the interval that ends first.

How this builds on earlier classes

The sorting of Class 10 often sets up a greedy scan. The backtracking of Class 24 explored every choice, while a greedy algorithm commits to one. The minimum-spanning-tree algorithms of Class 23 were greedy algorithms too.

Reading

- §7.22 Prim's Spanning Tree Algorithm, which is a greedy algorithm
- §4.12 Dynamic Programming, which shows a greedy rule failing on a coin-change example

Runestone has no chapter on greedy algorithms or intervals. For exchange arguments, read Chapter 4, "Greedy Algorithms," of Jeff Erickson's free textbook *Algorithms* (https://jeffe.cs.illinois.edu/teaching/algorithms/book/04-greedy.pdf), or ask your AI tool for a worked example. Ask your AI tool to explain a sweep over intervals as well.

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
455	Assign Cookies	Easy
134	Gas Station	Medium
435	Non-overlapping Intervals	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. What is your greedy rule, in one sentence? Then try to break it: write a small input on which a different choice would do better. If you cannot find one, explain why.
8. Once intervals are sorted by their start, write the test for whether two intervals (a, b) and (c, d) overlap.

Take-Home 27: Dynamic programming in one dimension

For Class 27. I hand this out at the end of Class 26, and it is due on paper at the start of Class 27. Plan on at least two hours.

The pattern: a one-dimensional table

Dynamic programming solves a problem by combining the answers to smaller versions of the same problem, and it stores each answer so that it is computed only once. You write the answer for state i in terms of earlier states, a formula called the recurrence. Then you choose the base cases and fill in the table in order. A one-dimensional table gives O(n) time and O(n) space, and you can often cut the space to O(1) by keeping only the last few values.

How this builds on earlier classes

The naive recursion of Class 9 repeated the same work many times. Dynamic programming is that recursion with a memory: either a dict, from Class 4, that stores answers as you find them, or a list, from Class 2, that you fill from the bottom up.

Reading

- §4.12 Dynamic Programming

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
70	Climbing Stairs	Easy
746	Min Cost Climbing Stairs	Easy
213	House Robber II	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. Define the state in words, in the form “dp[i] means …”. Then write the recurrence and the base cases.
8. Which earlier states does dp[i] need? Can you keep only those?

Take-Home 28: Dynamic programming on sequences

For Class 28. I hand this out at the end of Class 27, and it is due on paper at the start of Class 28. Plan on at least two hours.

The pattern: states that are positions in a sequence

When the state is a position in a sequence, the recurrence usually asks one of two questions: what is the best answer that ends exactly at position i, or what is the best answer that uses only the first i items? The longest increasing subsequence and the longest common subsequence are the standard examples. The running time is the number of states multiplied by the work for each state: $O(n^2)$ for the longest increasing subsequence computed with a simple scan, and $O(mn)$ for the longest common subsequence of two strings of lengths m and n.

How this builds on earlier classes

Class 27 introduced one-dimensional tables, in which each state depends on a few states just before it. Here a state can depend on every earlier state, and the longest common subsequence brings in a second sequence, which leads to the two-dimensional tables of Class 29.

Reading

- §4.12 Dynamic Programming, read again

Runestone explains the idea of dynamic programming but does not cover these problems. Use Chapter 3, “Dynamic Programming,” of Jeff Erickson’s free textbook *Algorithms* (https://jeffe.cs.illinois.edu/teaching/algorithms/book/03-dynprog.pdf), together with your AI tool.

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
139	Word Break	Medium
53	Maximum Subarray	Medium
152	Maximum Product Subarray	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. Define the state in the form “dp[i] means …”. Does it describe an answer that ends at position i, or one that uses the first i items?
8. How many states are there, and how much work does each one need? Multiply the two to get the running time.

Take-Home 29: Dynamic programming in two dimensions

For Class 29. I hand this out at the end of Class 28, and it is due on paper at the start of Class 29. Plan on at least two hours.

The pattern: a two-dimensional table

When a problem involves two sequences, or a grid, the state needs two indexes: dp[i][j] is the answer for the first i items of one input and the first j items of the other. Each cell depends on its neighbors above it and to its left. Time and space are both O(mn). A row often needs only the row above it, so you can usually cut the space to the length of one row.

How this builds on earlier classes

Class 28 met its first two-dimensional table in the longest common subsequence, and this class makes such tables the main subject. The backtracking of Class 24 pruned a search, while dynamic programming removes the repeated work instead.

Reading

- §4.12 Dynamic Programming, read again

For edit distance, use Chapter 3, “Dynamic Programming,” of Jeff Erickson’s free textbook *Algorithms* (https://jeffe.cs.illinois.edu/teaching/algorithms/book/03-dynprog.pdf). Use your AI tool for traces that show how a table fills in.

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
62	Unique Paths	Medium
64	Minimum Path Sum	Medium
221	Maximal Square	Medium

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. Define dp[i][j] in words, and write down the first row and the first column of the table.
8. Draw the table for a tiny input and fill in three cells by hand.

Take-Home 30: Choosing the pattern

For Class 30. I hand this out at the end of Class 29, and it is due on paper at the start of Class 30. Plan on at least two hours.

The pattern: every pattern, mixed

The final class introduces no new pattern. You will get two Hard problems with no topic label, and you must decide which structure and which algorithm to use, justify your choice, and work out the time, the space and the memory. Review the one-line idea behind every pattern in the course: arrays, hashing, two pointers, sliding windows, stacks, queues, linked lists, recursion, sorting, binary search, trees, balanced search trees, heaps, graph search, shortest paths, union-find, minimum spanning trees, backtracking, tries, greedy algorithms, intervals and dynamic programming.

How this builds on earlier classes

This class builds on every earlier class. Hard problems usually combine two patterns.

Reading

- §2.8 Summary of the chapter on algorithm analysis
- §5.13 Summary of the chapter on searching and sorting
- §6.20 Summary of the chapter on trees
- §7.23 Summary of the chapter on graphs

Practice problems on LeetCode

Do all three problems, in the order listed.

#	Problem	Level
42	Trapping Rain Water	Hard
76	Minimum Window Substring	Hard
23	Merge k Sorted Lists	Hard

Your plan

Choose one of the practice problems and write a plan for it on paper before you write any code. Your plan answers these six questions.

1. What does the problem ask, in your own words, and what is the size of the input, n?
2. What is the brute-force approach, meaning the simplest approach that tries every possibility, and what is its Big-O running time?
3. Which pattern will you use, and why does it fit this problem?
4. What are the steps of your solution, written in plain English rather than in code?
5. What running time and auxiliary space do you expect, and how many bytes of memory will your solution use when n is 100,000?
6. Which two edge cases must your solution handle? An edge case is an unusual input, such as an empty list, that a correct solution must still handle.

Bring your written plan to class. Phones and computers are locked away during class, so the plan must be on paper.

Two more questions about this pattern

7. For each practice problem, write the name of the pattern and the Big-O of the brute-force approach, without writing any code.
8. Write a one-page summary of the course's patterns. For each pattern, write down the clue in a problem's wording that tells you to use it.